Deep Learning

Elisa Ricci
a.y. 2025/2026
University of Trento

Davide Donà

March 2026

Notes Repository

Contents

1 Introduction **1**

1.1 Basic Definitions 1

1.2 Historical Background 1

2 Machine Learning Recap **6**

2.1 Why do we need Machine Learning 6

2.2 Machine Learning - Another definition 6

2.3 Types of learning 7

2.3.1 Supervised Learning 7

2.3.2 Unsupervised Learning 8

2.3.3 Reinforcement Learning 10

2.4 Designing a Learning System 10

2.5 Assumptions in Machine Learning 11

2.5.1 Empirical risk minimization 11

2.5.2 Assumption violations 11

2.5.3 Underfitting 11

2.5.4 Overfitting 12

2.6 How to choose the algorithm 12

3 Feedforward Neural Networks **13**

3.1 From Logistic Regression to Feedforward NN 13

3.1.1 Maximum Likelihood Estimation 13

3.1.2 Logistic Regression 14

3.2 Feedforward Neural Networks 14

3.2.1 Difference from Linear Models 14

3.2.2 How can we improve Linear Models? 15

3.3 Training a NN 15

3.4 Modelling Choices 15

3.5 Cost Functions 16

3.6 Output Units 16
3.7 Activation Functions 16
3.8 Architectures 16
3.9 Optimizers 16
3.10 Regularization 16

1 Introduction

In this section, we will briefly introduce the basic concepts of deep learning and its historical background.

1.1 Basic Definitions

AI Artificial Intelligence (AI) is a discipline that has the ambition to create machines that mimic human intelligence. It encompasses a wide range of techniques and approaches, including rule-based systems, expert systems, and machine learning. The ultimate goal of AI is to develop systems capable of learning from experience as effectively as humans do.

Machine Learning Machine Learning (ML) is a subset of AI that enables machines to learn from experience (i.e., data) to improve their performance on a specific task.

Deep Learning This is a subfield of Machine Learning that particularly focuses on **Artificial Neural Networks** (ANNs). The term 'deep' refers to the use of multiple layers in the network, which allows it to learn complex representations of data, with multiple levels of abstraction.

1.2 Historical Background

The concept of artificial neural networks dates back to the 1940s, with the work of Warren McCulloch and Walter Pitts, who proposed a simple model that attempted to emulate the behavior of biological neurons. This is regarded as the first single-layer perceptron model, which is a fundamental building block of deep learning.

Electronic Brain In the first perceptron model, the inputs form a vector of **binary features** that are simply summed together and passed through a **threshold function**, without any **weights** involved.

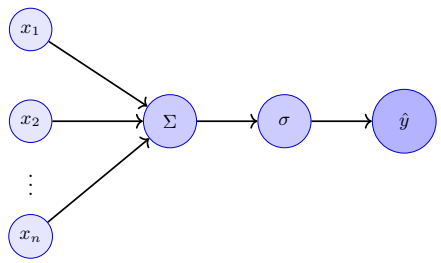

Figure 1: Architecture of the electronic brain model. Inputs x_i are summed together and passed through an activation function σ to produce the output $\hat{y}$.

It can be shown that this model is capable of classifying linearly separable data, such as the AND, OR, and NOT functions, but it fails to classify non-linearly separable data, such as the XOR function.

The Perceptron In the 1950s, Frank Rosenblatt introduced the **Perceptron**, which is an extension of the electronic brain model. The key difference is the introduction of **weights** for each input, which allows the model to learn from data and adjust its parameters accordingly.

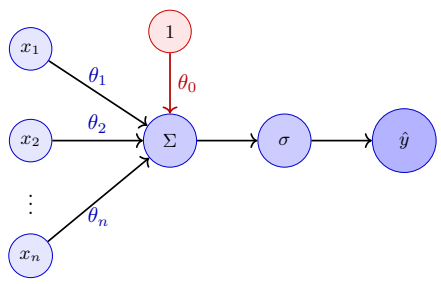

Figure 2: Architecture of a single layer perceptron. Inputs x_i are weighted by w_i, summed together, and passed through an activation function σ to produce the output y.

The prediction $\hat{y}$ of the perceptron is given by the following formula:

$$\hat{y} = \sigma\left(\sum_{i=1}^{n} \theta_i x_i + \theta_0\right) \tag{1.1}$$

where θ_i are the weights associated with each input x_i, and σ is the activation function. We can also express the prediction in vector form as follows:

$$\hat{y} = \sigma\left(\boldsymbol{\theta}^T \boldsymbol{x} + \theta_0\right) \tag{1.2}$$

The most important intuition is that the weights θ_i are not fixed, but rather learned from data.

The bias term is introduced to allow the model to shift the decision boundary (hyperplane) away from the origin.

Perceptron Learning Algorithm The weights of the perceptron are learned from data using the **Perceptron Training Rule**. Given a set of training examples $\mathcal{T} = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots, (\boldsymbol{x_N}, y_N)\}$, $y_i \in \{0, 1\}$ and a learning rate η, the weights are updated as follows:

1. Initialize the weights θ_i to small random values;
2. Until convergence, repeat:
   (a) Select randomly a training example $(\boldsymbol{x_i}, y_i)$ from $\mathcal{T}$;
   (b) Compute the prediction $\hat{y}_i$ using the current weights:
   $$\hat{y}_i = \sigma\left(\boldsymbol{\theta}^T \boldsymbol{x_i} + \theta_0\right)$$
   (c) Update the weights based on the error between the prediction and the true label:
   $$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} + \eta(y_i - \hat{y}_i)\boldsymbol{x_i}$$
   (d) Update the bias:
   $$\theta_0^{(t+1)} \leftarrow \theta_0^{(t)} + \eta(y_i - \hat{y}_i)$$

Note that we are assuming that the training data is linearly separable, otherwise we would have to introduce a stopping criterion to prevent the algorithm from running indefinitely.

Perceptron Today Today, the activation function σ is typically a non-linear function, such as the sigmoid, ReLU, or tanh function, which allows the model to learn complex patterns in the data.

Perceptron

An artificial neuron, i.e., a perceptron is a non-linear parametrized function with a restricted output range.

Perceptron Problem As anticipated, in the 1970s it was found that the perceptron was not able to learn non-linearly separable data, such as the XOR function. This was seen as a major setback for the field of AI, and it led to a decline in interest and funding for research in this area, which is now known as the "AI winter".

MLP AI did not stop there, and in the 1980s, researchers introduced the concept of **Multilayer Perceptron** (MLP). This architecture had a major issue, which was the difficulty in computing the weights of the hidden layers.

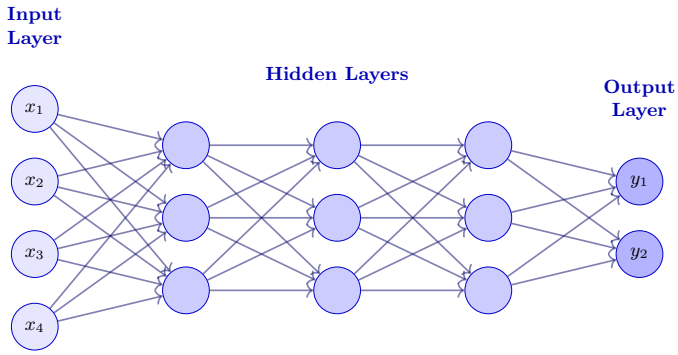

Figure 3: Architecture of a Multi-Layer Perceptron (MLP) with three hidden layers, fully connected in a feed-forward fashion.

This structure was very flexible as multiple nodes could be used in the output layer, so it could be used for multi-class classification problems, and also for regression.

Backpropagation The perceptron learning algorithm was not suitable for training MLPs, as no expected output was available for the hidden layers, but just for the output layer.

Backpropagation is an efficient algorithm for computing the gradients also for large datasets. At a high level, the algorithm can be summarized as follows:

1. **Forward propagation**: Given an input $\boldsymbol{x}$, compute the output of the network $\hat{y}$ by propagating the input through the layers of the network;
2. **Loss computation**: Compute the loss $L(y, f(\boldsymbol{x}, \boldsymbol{\theta}))$, where y is the true label and $f(\boldsymbol{x}, \boldsymbol{\theta})$ is the output of the network with parameters $\boldsymbol{\theta}$;
3. **Backward propagation**: the error signal is propagated backwards through the network and it's used to update the weights.

Obviosly, this concept will be covered in more detail in the next chapters.

Other Architectures Other architectures were proposed in the following years that lead to the current breakthrough of deep learning, such as:

- **Convolutional Neural Networks** (CNNs), which are particularly effective for image processing tasks;

- **Recurrent Neural Networks** (RNNs), used for sequential data, such as time series and natural language processing;

Cybenko's Theorem Cybenko's theorem states that a 2-layer feed-forward neural network with linear output can approximate any continuous function on a compact subset of $\mathbb{R}^n$ with arbitrary accuracy.

In practice, the more hidden layers we have, the more complex functions we can learn. Deep neural networks need a lot of labeled data to be trained, and cannot be trained easily.

Second Winter of AI Three main factors contributed to the second AI winter in the late 1980s:

- **Deep Neural Networks** tend to overfit the training data, and had problems with vanishing gradients, which made them difficult to train;
- **Lack of computational resources** to train deep neural networks. In fact, no GPUs were available at the time;
- **Lack of large datasets** to train deep neural networks.

All these issues led to a decline in interest and funding for research in this area, with an increase of interest in other areas of ML, such as Support Vector Machines (SVMs) and Kernel Methods.

First Deep Learning Applications - AlexNet In 2012, a deep convolutional neural network called AlexNet won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) with a top-5 error rate of 15.3%, which was significantly better than the second-best entry, which had an error rate of 26.2%. This was a major breakthrough for the field of deep learning, and it led to a shift of interest and funding back to this area, which is now known as the "Deep Learning Revolution".

AlexNet also shifted the focus of research from shallow architectures to deep architectures. The first one required heavy works on feature engineering before training the model, while the second one was able to learn features directly from the data, without any human intervention.

2 Machine Learning Recap

Machine Learning is a subset of AI, born in contrast to the programming paradigm, which is based on the idea of writing explicit rules to solve a problem.

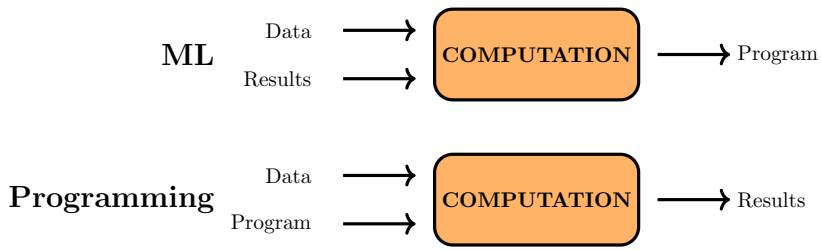

Figure 4: Programming paradigm vs Machine Learning paradigm.

In the Machine Learning paradigm, we are instead given the data, along with the results, and we try to learn the **model**. The learned model (e.g. splits in a decision tree, linear function for SVMs) represents the solution to the problem, and it can be used to make predictions on new data.

2.1 Why do we need Machine Learning

The main reason why we need Machine Learning can be summarized in the following points:

- **Complexity of the problem**: For many tasks, we have no human expertise to write explicit rules;
- **Personalized solutions**: For many tasks, we need to learn personalized solutions for each user, which is not feasible with the programming paradigm;
- **Huge amount of data**: Working with huge amount of data is not feasible, as it would require writing down explicit rules for each possible combination of inputs.

2.2 Machine Learning - Another definition

Machine Learning

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at

tasks in T, as measured by P, improves with experience E.

This definition is more formal and precise than the previous one, and it highlights the importance of the three components: experience, task, and performance measure. Each machine learning algorithm can be characterized by these three components. For example, taking in consideration a spam email filter, we can define:

- T: Categorizing emails as spam or not spam;
- P: Accuracy of the classification. Other metrics may be more appropriate;
- E: Database of emails labeled as spam or not spam by human experts.

2.3 Types of learning

We can identify three main types of learning:

- **Supervised learning**: We are given a training set of input-output pairs, and we want to learn a function that maps inputs to outputs.
- **Unsupervised learning**: We are given a training set of inputs, and we want to learn a function that captures the underlying structure of the data.
- **Reinforcement learning**: We are given an environment, and we want to learn a policy that maximizes the cumulative reward.

2.3.1 Supervised Learning

Supervised learning can be further divided into subcategories, such as:

- **Classification**: The output is a discrete label. More formally, given a training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ we can define it as:

$$f : \mathcal{X} \to \{1, \ldots, k\} \tag{2.1}$$

  where $\mathcal{X}$ is the input space, and $\{1, \ldots, k\}$ is the set of possible labels.

  Based on the number of classes, we can further divide classification into **binary classification** and **multi-class classification**.

- **Regression**: The output is a continuous value. More formally, given a training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ we can define it as:

$$f : \mathcal{X} \to \mathbb{R} \tag{2.2}$$

- **Structured output**: **supervised learning** doesn't necessarily require a **vector** as input, and a **scalar** as output. We can have more complex input and output structures, such as images, graphs, or sequences. In all cases the output is a structured object, with relationships between its components. Some practical example of **structured input regression** problems are:
  - **Semantic segmentation**: Given an image of input size $M \times N \times 3$, we want to predict a mask of size $M \times N$, where each pixel is labeled with a class. The original solution to this kind of problem was to use **feature engineering** to extract features from neighborhoods of pixels. After that, SVMs were used to make predictions based on the extracted features. **Neural Networks** show their real power in this kind of problems, as they are able to learn features directly from the data, without any human intervention.
  - **Machine translation**: given a sentence (sequence of words) in a source language, we want to translate it to a target language. This is even more complex than the previous example, as not only the output is related to its adjacent components, but also grammatical and semantic relationships between words need to be captured.

2.3.2 Unsupervised Learning

In unsupervised learning, we are given a training set of inputs, without any corresponding output labels, and we want to learn the underlying structure of the data. Some examples of unsupervised learning tasks are:

- **Clustering**: grouping similar data points together;
- **Deep clustering**: a technique that combines deep learning with classical clustering algorithms, such as k-means, to learn better representations of the data and improve clustering performance. It is based on the idea of using a deep neural network to learn a low-dimensional representation of the data. This is then used as input to a clustering algorithm, producing a set of **pseudo-labels** that are used to train the neural network using backpropagation. This process is repeated iteratively until convergence;

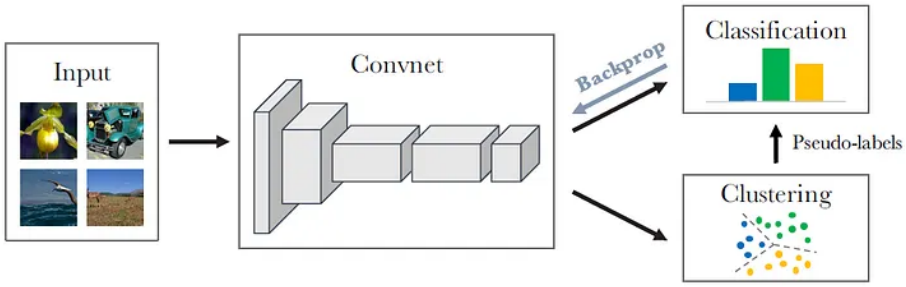

Figure 5: Deep clustering.

- **Generative Models**: models learn the underlying distribution of the data, and can be used to generate new data points, similar to the ones in the training set. Also in this case, we can have **deep generative models**. They try to map exaples extracted from a simple distribution Z to a more complex distribution $g_\theta(Z)$, which is similar to the distribution of X.

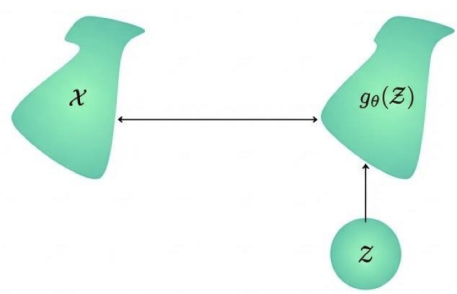

Figure 6: Deep generative models.

- **Dimensionality reduction**: try to map data to a lower-dimensional space in which visualization and interpretation are easier. This is often used as a preprocessing step for other machine learning algorithms.
- **Autoencoders**: a type of encoder-decoder architecture, where:
  - The **encoder** maps the input data to a **latent space** of lower dimensionality;
  - The **decoder** maps the latent representation back to the original input.

  The two components are trained together to minimize the reconstruction error, which is the difference between the original input and the reconstructed output. Once trained, the encoder can be used alone to obtain lower-dimensional representations of the data.

2.3.3 Reinforcement Learning

In Reinforcement Learning, an **agent** learn to interact with an **environment** in order to maximize a cumulative reward. We are not given any data, but rather the agent learns from its own experience, by exploring the environment and receiving feedback in the form of rewards.

In **deep reinforcement learning** the agent uses a deep neural network to learn a policy that maps states of the environment to actions. This allows the agent to learn complex behaviors and solve tasks that are difficult to solve with traditional reinforcement learning algorithms.

2.4 Designing a Learning System

To design a learning system, it's important to consider the following steps:

1. **Choose the training experience** E: This is the data that will be used to train the model. It should be representative of the problem we want to solve, and it should be of good quality;
2. **Choose the model**: This is the function that we want to learn from the data. It heavily depends on the task we want to solve,
3. **Choose how to represent the model**: the form of the model, what kind of function we allow. For example, we can have:
   - **Instance-based functions**: predictions are made based on the similarity between the input and the training examples. For example, k-nearest neighbors;
   - **Numerical functions**: predictions are made based on a mathematical function that maps inputs to outputs. For example, Neural Networks, SVMs;
   - **Probabilistic functions**: predictions are made based on a probability distribution over the possible outputs. For example, Naive Bayes;
   - **Symbolic functions**: predictions are made based on a set of rules that are learned from the data. For example, decision trees.
4. **Choose the learning algorithm**: This is the algorithm that will be used to learn the model from the data. For example, for SVMs we can decide between different optimization algorithms, such as stochastic gradient descent or quadratic optimization.

   The success of the machine learning system also depends on the search and optimization algorithm used to find the best model.

5. **Choose the performance measure** P: This is the metric that will be used to evaluate the performance of the model. Different tasks require different performance measures and it's not always a trivial choice. Ideally we would like to choose a performance measure that is closely related to the loss function used to train the model.

2.5 Assumptions in Machine Learning

When developing a machine learning model, we usually assume that we are given a training set of examples that are **observed** and we want to learn a model that can make predictions on new, unseen examples, the **test set**.

The challenge is that the algorithm has to learn a model that performs well on new examples. This ability is known as **generalization**.

2.5.1 Empirical risk minimization

Classical optimization algorithms are different from machine learning algorithms. In classical optimization, the only goal is to find the best solution to a problem, reducing the **training error** as much as possible. In machine learning instead, we want to minimize the **test error**, which is the error on new, unseen examples. This is a much harder problem, as we don't have direct access to the test set during training.

To tackle this problem, we assume that both training and test examples are drawn from the same distribution. This is known as the **independent and identically distributed** (i.i.d.) assumption, and it allows us to use the training set to learn a model that generalizes well to the test set.

If the training set is representative of the underlying unknown distribution, then the **empirical loss** (the average loss on the training set) is a good estimate of the **expected loss** (the average loss on the test set). This is the principle of **empirical risk minimization**.

2.5.2 Assumption violations

If the training set is not representative of the underlying distribution or if the i.i.d. assumption is violated, then the model may not generalize well to the test set, and we may end up with a model that performs well on the training set but poorly on the test set. This is known as **overfitting**.

2.5.3 Underfitting

If the model is too simple, it may perform poorly on both the training and test set. This is known as **underfitting**. To avoid underfitting, we need to choose

a more complex model, or to use a more powerful learning algorithm.

2.5.4 Overfitting

If the model is too complex, it may perform well on the training set but poorly on the test set. When the gap between the two errors is large, it means that the model has learned to fit the noise in the training data, rather than the underlying distribution. This is known as **overfitting**.

To avoid overfitting, we can use techniques such as:

- **Simpler models**: choosing a simpler model with fewer parameters, which is less likely to overfit the training data;
- **Regularization**: any modification made to a learning algorithm that is intended to reduce the generalization error, but not the training error. Usually, this is done by adding a penalty term to the loss function, which is proportional to the complexity of the model (e.g., adding a term that penalizes large weights in a neural network).

2.6 How to choose the algorithm

To choose the right algorithm for a given problem, we need to consider the following factors:

- **Size of the training set**: Some algorithms perform better with small training sets, while others require large training sets to perform well;
- **Address problem difficulty**: are data linearly separable? Is the problem a regression or a classification problem? Is the output structured?
- **Prior knowledge**: if prior knowledge about the problem is available, it can be encoded in the model, which can improve performance;
- **Computational requirements**: do we have any requirements on the computational resources available for training and inference?
- **Online vs batch learning**: based on the kind of data we have, we may need to choose different algorithms. For example, for streaming data, Neural Network aren't the best choice;
- **Interpretability**: do we need to understand how the model makes predictions?

3 Feedforward Neural Networks

Before we can understand the architecture of a **feedforward neural network** (FFNN), we first introduce how we got here.

3.1 From Logistic Regression to Feedforward NN

To fit a model to some data distribution, a simple and common approach is to use **Maximum Likelihood Estimation** (MLE).

3.1.1 Maximum Likelihood Estimation

In MLE, we want to estimate the parameters of an assumed probability distribution, given some observed data. For example, if we assume that the data is generated from a Gaussian distribution, we can estimate the mean and variance of that distribution using MLE.

More formally, given a dataset $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ and a known probability distribution $p_{\text{model}}(\boldsymbol{x}|\boldsymbol{\theta})$ we want to find the parameters $\boldsymbol{\theta}$ that better explain the data.

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) \tag{3.1}$$

Assuming that the data points are independent and identically distributed (i.i.d.), we can express the likelihood as a product of individual probabilities:

$$p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = \prod_{i=1}^{N} p_{\text{model}}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

To simplify the optimization process, we often take the logarithm of the likelihood function, which transforms the product into a sum:

$$\log p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p_{\text{model}}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

We can show that this is equivalent to considering the expected value of the log-likelihood over the empirical (observed) data distribution:

$$\log p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = N \cdot \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}[\log p_{\text{model}}(\boldsymbol{x}|\boldsymbol{\theta})]$$

where p_{data} is the empirical distribution of the observed data and $\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}$ denotes the expected value sampled from this distribution.

To find the optimal parameters $\boldsymbol{\theta}^*$, we usually differentiate the log-likelihood with respect to $\boldsymbol{\theta}$ and set it to zero. We can then resolve the resulting equation to find the optimal parameters.

The equation 3.1 can also be extended to the case of conditional probability distributions ($p_{\text{model}}(y|\boldsymbol{x}, \boldsymbol{\theta})$) in order to predict a target variable y given an input variable $\boldsymbol{x}$:

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} p_{\text{model}}(y|\boldsymbol{x}, \boldsymbol{\theta})$$

This is the basis for most supervised learning algorithms, including Linear Regression and Logistic Regression, which simply assume different forms for the conditional probability distribution. For example, in Logistic Regression, we assume that the conditional probability distribution is given by the logistic function:

$$p_{\text{model}}(y = 1|\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T \boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \boldsymbol{x}}} \tag{3.2}$$

3.1.2 Logistic Regression

Logistic Regression is a linear model used for binary classification tasks. Given the equation 3.2, we can observe that the can be seen as a simple Neural Network with a single layer and a sigmoid activation function.

3.2 Feedforward Neural Networks

A Feedforward Neural Network (FFNN) is a type of artificial neural network that define a mapping from the input space to an output space. The learned mapping $f(\boldsymbol{x})$ is defined as a composition of several functions, each representing a layer of the network. For example, taking in consideration the FFNN architecture shown in the figure 3 with three layers, we can express the mapping as follows:

$$f(\boldsymbol{x}) = f^{(3)(f^{(2)}(f^{(1)}(\boldsymbol{x})))}$$

where $f^{(1)}$, $f^{(2)}$, and $f^{(3)}$ represent the functions corresponding to the first, second, and third layers of the network, respectively.

The last layer of the network is often referred to as the **output layer**, while the layers before it are called **hidden layers**. The function composition can be described by a **directed acyclic graph** (DAG), where the nodes represent the functions (layers) and the edges represent the flow of data through the network. The term "feedforward" comes from the fact that the data flows in one direction, from the input layer to the output layer, without any cycles or loops in the graph.

3.2.1 Difference from Linear Models

Linear models are generally very easy to optimize (e.g., closed-form solutions or convex optimization), but they are limited in their expressiveness and cannot capture interactions between features or non-linear relationships in the data.

FFNNs allow to overcome the limitations of linear models by introducing non-linear activation functions and multiple layers of computation.

3.2.2 How can we improve Linear Models?

FFNNs are not the only way to improve linear models, but they are one of the most powerful and widely used approaches. Generally, to boost the performance of linear models, we can apply a non-linear transformation to the input data $\boldsymbol{x} \to \phi(\boldsymbol{x})$, where ϕ is a non-linear function. We can then apply a linear model to the transformed data $\phi(\boldsymbol{x})$. The transformation ϕ can be implemented in various ways, such as:

- **Kernel Methods**: map the input data into a high-dimensional feature space (possibly infinite-dimensional) where linear models can be applied. This has enough capacity to capture complex relationships in the data, but achieves poor generalization performance for highly variable target functions.
- **Feature Engineering**: manually design and select features that capture relevant information from the input data. This was the traditional approach before the rise of deep learning, in particular for computer vision tasks, but it is time-consuming and requires domain expertise.
- **Neural Networks**: learn the transformation ϕ from the data itself. This shifts the engineering effort from designing features to designing the architecture of the network, but throwhs away the convexity and interpretability.

3.3 Training a NN

To train a Neural Network, we need to optimize the parameters of the network (weights $\boldsymbol{\theta}$) to drive the output of the network $f(\boldsymbol{x}; \boldsymbol{\theta})$ to match the target variable y for a given input $\boldsymbol{x}$.

The training process is nothing different from training any other machine learning model. We need to define a loss function $\mathcal{L}(f(\boldsymbol{x}; \boldsymbol{\theta}), y)$ that measures the discrepancy between the predicted output and the true target variable.

The largest difference with respect to linear models comes from the fact that most loss functions are non-convex, which removes the guarantee of finding a global minimum and makes the optimization process more challenging.

3.4 Modelling Choices

When designing a FFNN, we need to make several modelling choices, such as:

- **Cost Function**: the choice of the loss function $\mathcal{L}$ depends on the type of problem we are trying to solve (e.g., regression, classification) and the properties of the data. Each function has its own advantages and disadvantages, and the choice can significantly impact the performance of the model.
- **Form of Output**:
- **Activation Function**:
- **Architecture**:
- **Optimizer**:
- **Regularizer**:

3.5 Cost Functions

3.6 Output Units

3.7 Activation Functions

3.8 Architectures

3.9 Optimizers

3.10 Regularization